

Modern Computer Vision Architecture

Backbones, Heads and Pretraining

Amit Kumar Saha (A00336661)
19/04/2026

Contents

1. Introduction	1
Example 1: ResNet as a shared backbone	1
Example 2: Vision Transformer backbones reused across tasks	1
2. Supervised and Self-supervised pretraining.....	1
2.1 Supervised pretraining	2
Strategy 1: ImageNet classification pretraining	2
Strategy 2: Supervised detection pretraining on COCO	2
2.2 Self-supervised pretraining	2
Strategy 1: Siamese / contrastive learning	2
Strategy 2: DINO-style self-distillation.....	3
2.3 Comparison - Supervised and Self-Supervised Pretraining	3
3. Case study: SAM as backbone + head	4
4. Limitations - Generic backbones	4
Conclusion	5
References	6

Figures

Figure 1 Backbone + Head Design Pattern in Vision Models	1
Figure 2 Supervised pretraining flow.....	2
Figure 3 Self-supervised pretraining flow	3
Figure 4 SAM as Backbone + Head + Prompts	4
Figure 5 Tasks - Recognition vs Reconstruction	5

1. Introduction

Modern vision models are typically modular: a **backbone** extracts reusable visual features, and a **head** maps those features to a task output. Early backbone layers capture local patterns (edges/textures); deeper layers capture semantic structure. The same backbone can therefore support multiple downstream tasks by replacing or fine-tuning the head.

This separation improves transfer learning efficiency: instead of retraining full models for each task, we reuse one pretrained representation and attach task-specific heads.

Backbone, in simple terms, is the main feature extractor shared across tasks.

Head, in simple terms, is the task-specific predictor attached to backbone features (classifier, detector, segmenter, depth regressor, etc.).

Example 1: ResNet as a shared backbone

ResNet was introduced for **ImageNet** classification [1], then reused in detection and segmentation pipelines such as **Faster R-CNN** and **Mask R-CNN** [2,3]. The backbone remains similar and only the head changes with task requirements.

Example 2: Vision Transformer backbones reused across tasks

ViT backbones are used for classification, detection, segmentation, and depth tasks [4]. DINO-style self-supervised ViTs are similarly adapted with different heads for downstream tasks [5].

The backbone + head idea is illustrated with a simple example of an image classification model in Figure 1:

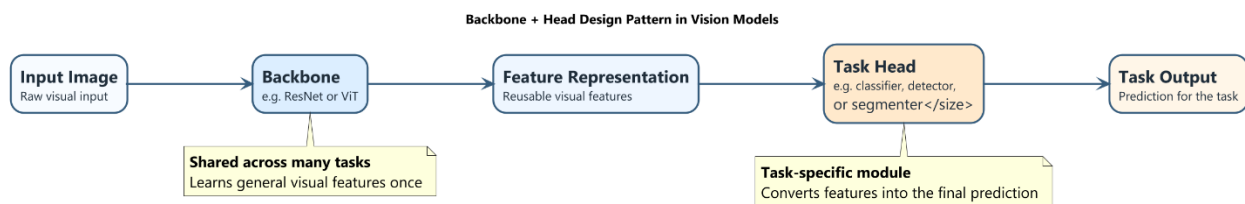


Figure 1 Backbone + Head Design Pattern in Vision Models

Figure 1 highlights the modular structure: the backbone extracts reusable features, and the head converts them into the output needed for a specific task.

2. Supervised and Self-supervised pretraining

Pretraining initializes a backbone before task-specific adaptation. The key difference is the supervision source.

2.1 Supervised pretraining

In supervised pretraining, learning is guided by human annotations.

Strategy 1: ImageNet classification pretraining

A standard supervised strategy is to train a backbone for image classification on **ImageNet**, as done for models such as **VGG** and **ResNet** [1, 6]. The model takes an image and predicts one class label from a fixed taxonomy.

Training signal: image-level class labels.

Features encouraged: category-level semantic discrimination and invariance to nuisance factors.

Strategy 2: Supervised detection pretraining on COCO

Another supervised strategy is to train a backbone jointly with a detection head on an object detection dataset such as **COCO**, for example in **Faster R-CNN** or **Mask R-CNN** [2, 3].

Training signal: object classes + bounding boxes (and masks for Mask R-CNN).

Features encouraged: object-level, spatially localized representations useful for recognition + localization.

Supervised pretraining is strongly aligned with labeled objectives but depends on annotation quality and coverage.

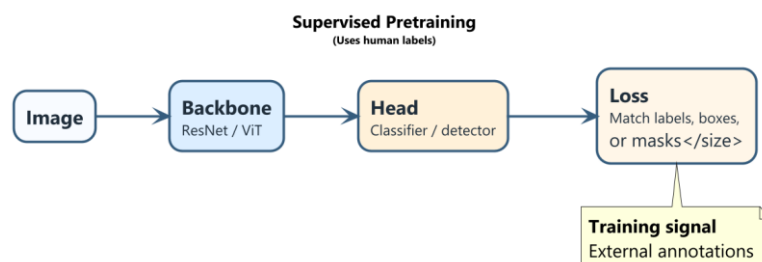


Figure 2 Supervised pretraining flow

2.2 Self-supervised pretraining

Self-supervised learning (**SSL**) derives supervision from data structure rather than manual labels.

Strategy 1: Siamese / contrastive learning

A widely used self-supervised pretraining strategy is **Siamese** or **contrastive learning**, including **SimCLR** and **MoCo** [7, 8]. The core idea is to create two different augmented views of the same image, for example by random cropping, colour jittering, or flipping. These two views should produce similar representations for the same underlying image and different representations for different images.

Training signal: positive pairs (two views of same image) are pulled together; negatives (different images) are pushed apart.

Features encouraged: invariance to augmentations and grouping by semantic similarity.

Strategy 2: DINO-style self-distillation

DINO is another influential self-supervised pretraining strategy based on self-distillation [5]. It uses a student network and a teacher network. Both receive different views of the same image. The teacher produces a target distribution over features, and the student learns to match it. The teacher itself is updated from the student using an exponential moving average, so no human labels are needed.

Training signal: student matches teacher outputs for different views without labels.

Features encouraged: robust view-invariant features and emergent object-aware structure.

SSL can scale to large unlabeled corpora and often yields transferable representations.

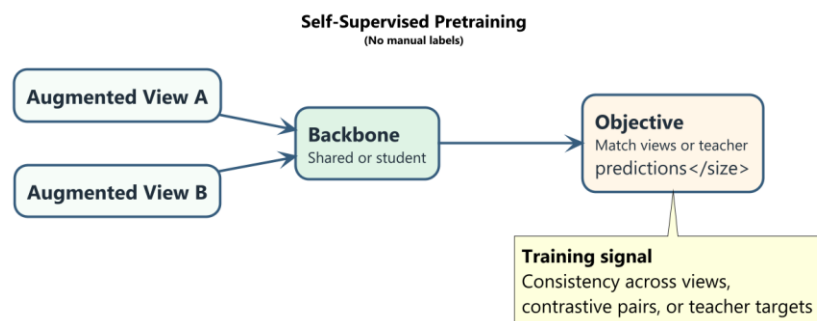


Figure 3 Self-supervised pretraining flow

2.3 Comparison - Supervised and Self-Supervised Pretraining

Key difference between Supervised and Self-Supervised Pretraining is in supervision source. While supervised pretraining relies on external annotations such as class labels or boxes SSL derives supervision from the data itself (by matching/contrasting views or student-teacher consistency).

Supervised pretraining is often more label-space aligned and SSL is often broader and less taxonomy-bound, with strong transfer potential.

Although SSL scales with abundant unlabelled data and lowers annotation dependence, its pretraining objective may be imperfectly aligned with downstream tasks, requiring stronger adaptation/fine-tuning.

3. Case study: SAM as backbone + head

Segment Anything Model (**SAM**) is a clear backbone-head system while adding prompts as an important interface [9]:

The **image encoder** acts as a reusable backbone. The **prompt encoder** converts points/boxes/masks into conditioning signals and the **mask decoder** is the task head that predicts masks conditioned on prompts.

The role of the point/box/mask prompts is to specify what to segment in the same image representation space. By changing prompts (and usage mode), SAM can perform different segmentation-like behaviours without retraining the full backbone.

Conceptually, SAM demonstrates the principle “train a strong backbone once, then reuse it across many segmentation-like tasks via prompt-conditioned heads/decoding.”.

Figure 4 shows SAM using backbone, head and prompt for image processing.

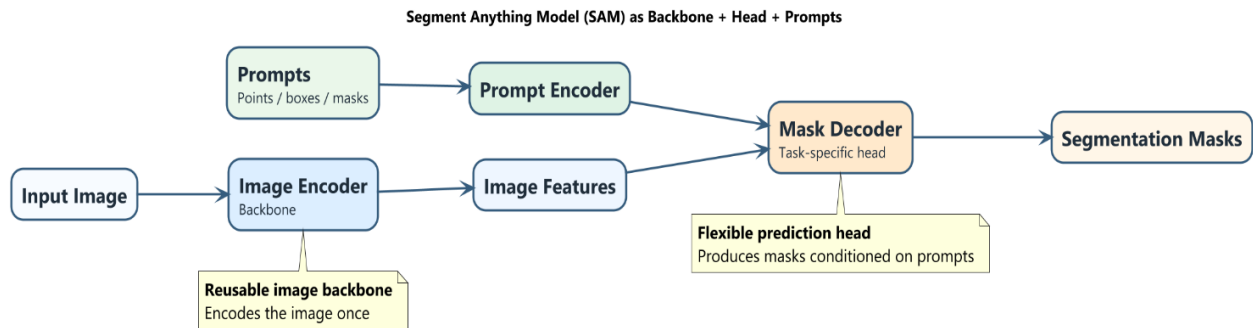


Figure 4 SAM as Backbone + Head + Prompts

4. Limitations - Generic backbones

Generic recognition backbones are not universally optimal. A representative limitation is image super-resolution. Super-resolution requires precise pixel-level reconstruction (textures, edges, local correspondences). Recognition-oriented backbones are often trained for semantic invariance, which can suppress the fine detail required for restoration.

High-level invariant features may be insufficient as the task is not only “what is in the image” but “what exact pixel detail is missing”. Invariance helpful for recognition can remove information needed for faithful reconstruction.

Hence, fine-grained pixel-level structure and local correspondence, inductive bias for restoration/degradation inversion and task-specific objectives for perceptual and reconstruction fidelity is needed instead.

Figure 5 compares high-level *recognition tasks* with low-level *reconstruction tasks*. Recognition models benefit from semantic, invariant features, while reconstruction tasks such as super-resolution need precise pixel-level detail and spatial accuracy. It highlights why generic backbones are often less suitable for restoration problems.

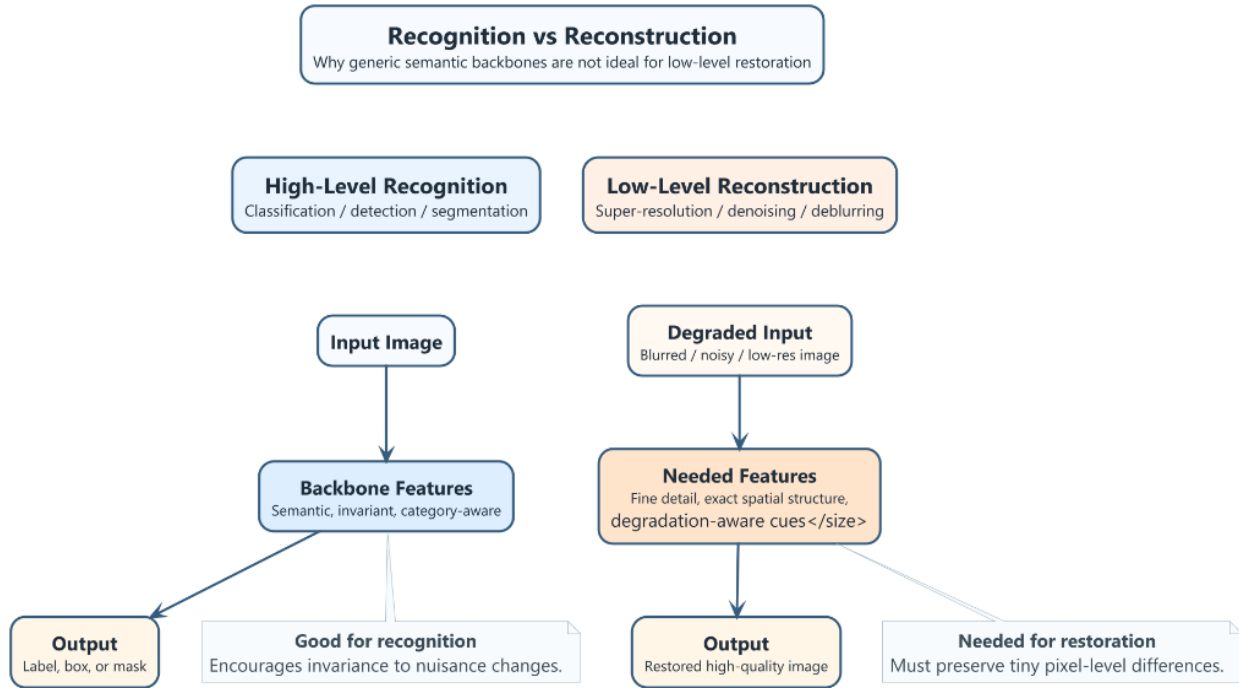


Figure 5 Tasks - Recognition vs Reconstruction

Conclusion

The **backbone-head paradigm** is central to modern vision: backbones provide transferable representations, and heads provide task specialization. Both supervised and self-supervised pretraining fit this paradigm, with different strengths in label dependence, scalability, and objective alignment.

SAM demonstrates the practical power of this design by combining a shared backbone with prompt-conditioned decoding. However, the same modular strategy has limits: low-level reconstruction tasks often require task-specific inductive biases beyond generic semantic backbones.

References

- [1] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," 2016.
- [2] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks," 2015.
- [3] K. He, G. Gkioxari, P. Dollar, and R. Girshick, "Mask R-CNN," 2017.
- [4] A. Dosovitskiy et al., "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale," 2021.
- [5] M. Caron et al., "Emerging Properties in Self-Supervised Vision Transformers," 2021.
- [6] K. Simonyan and A. Zisserman, "Very Deep Convolutional Networks for Large-Scale Image Recognition," 2015.
- [7] T. Chen et al., "A Simple Framework for Contrastive Learning of Visual Representations," 2020.
- [8] K. He, H. Fan, Y. Wu, S. Xie, and R. Girshick, "Momentum Contrast for Unsupervised Visual Representation Learning," 2020.
- [9] A. Kirillov et al., "Segment Anything," 2023.